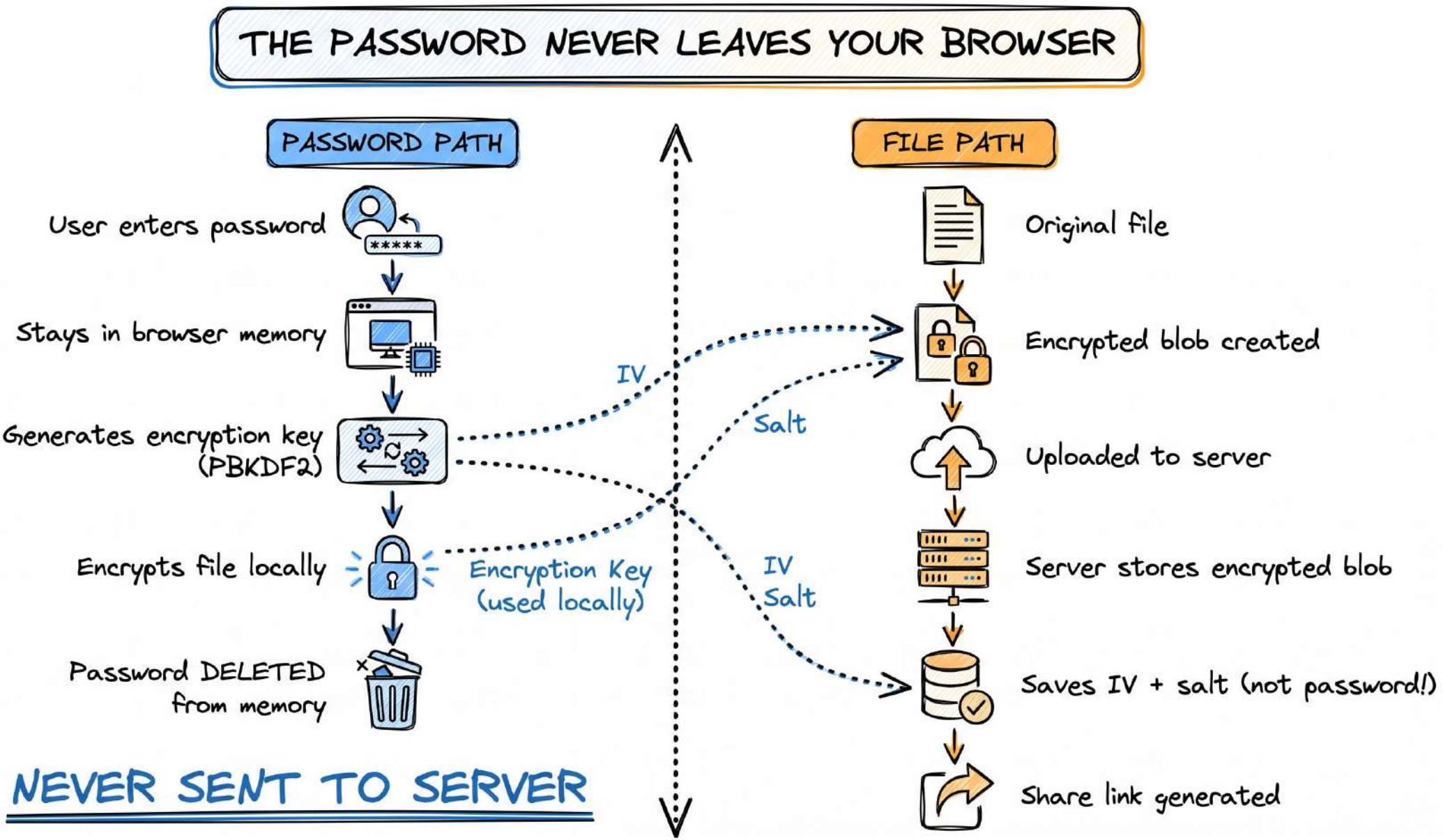
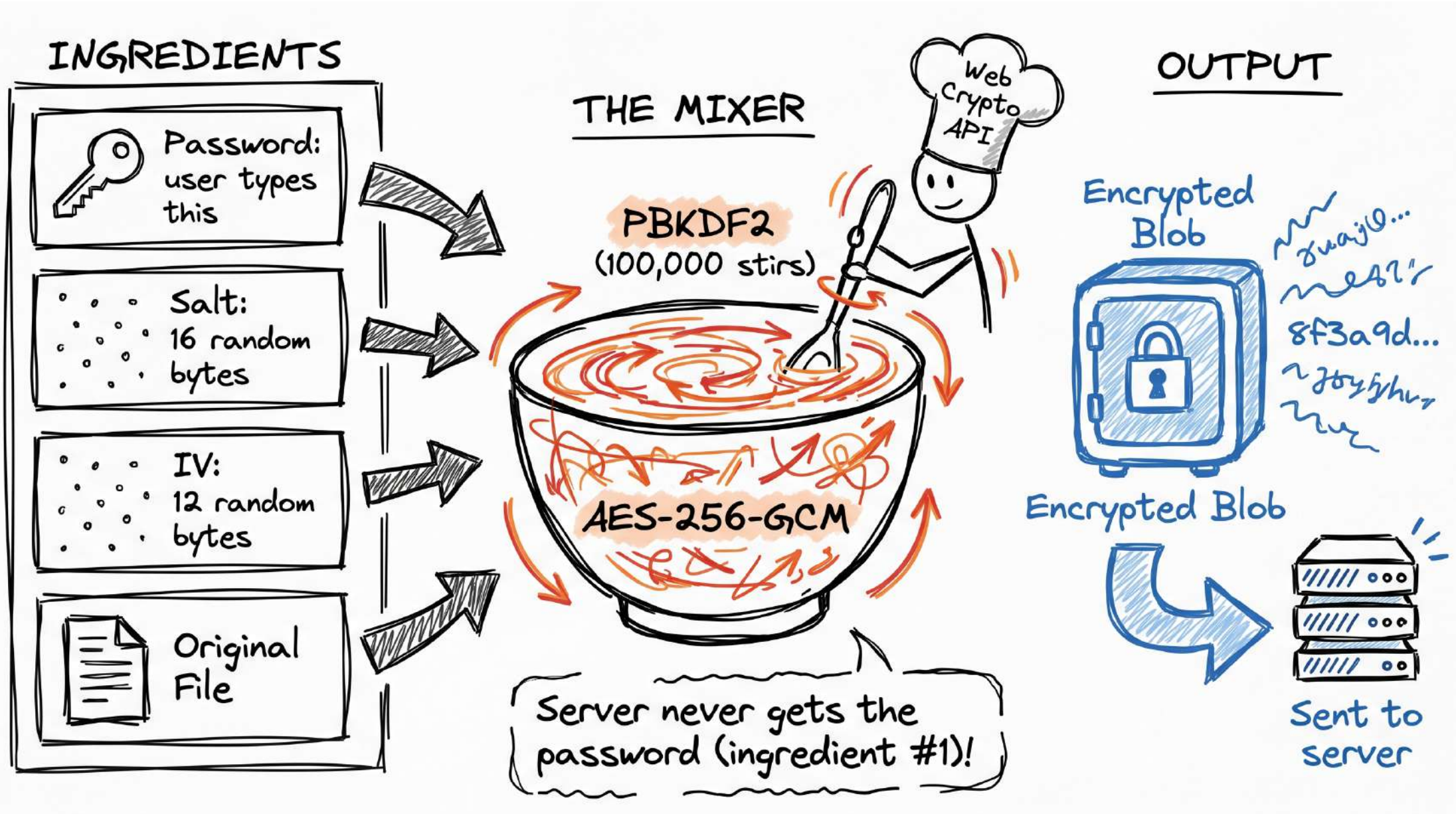
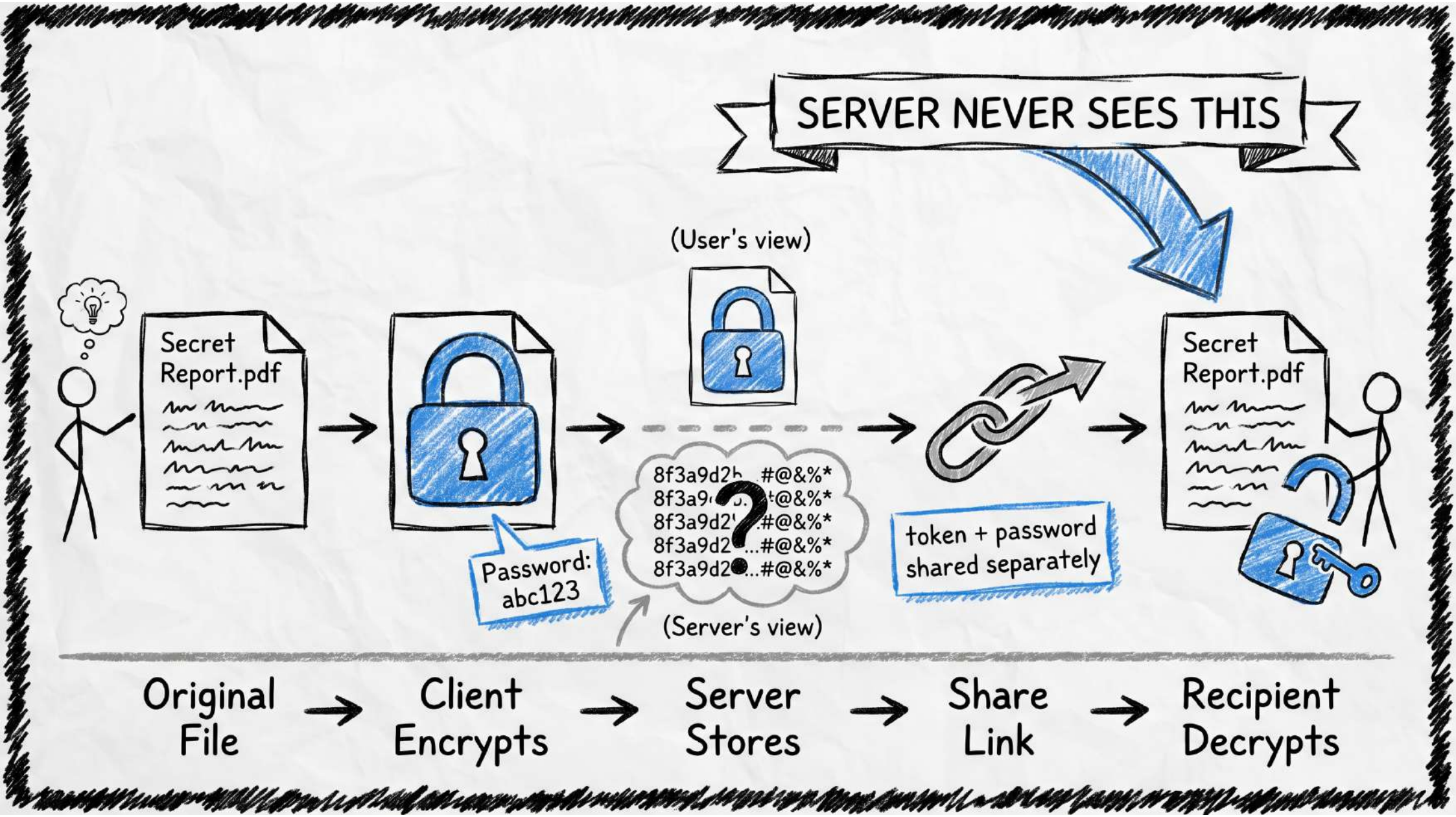

THE PASSWORD NEVER LEAVES YOUR BROWSER
PASSWORD PATH
FILE PATH
User enters password
*****
Stays in browser memory
Generates encryption key (PBKDF2)
Encrypts file locally
Password DELETED from memory
NEVER SENT TO SERVER
IV
Salt
Encryption Key (used locally)
IV
Salt
Original file
Encrypted blob created
Uploaded to server
Server stores encrypted blob
Saves IV + salt (not password!)
Share link generated

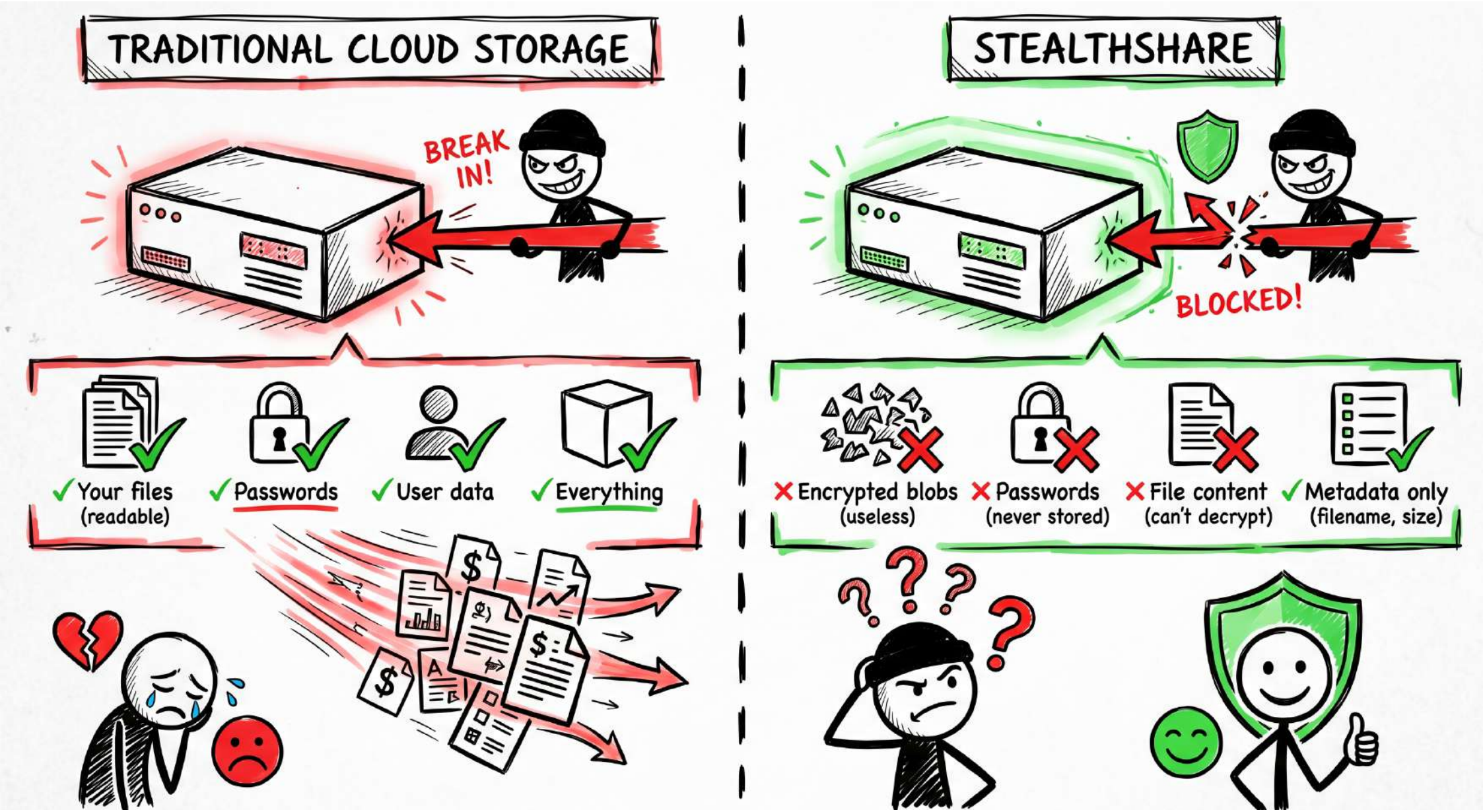

INGREDIENTS
Password: user types this
Salt: 16 random bytes
IV: 12 random bytes
Original File
THE MIXER
Web Crypto API
PBKDF2
(100,000 stirs)
AES-256-GCM
Server never gets the password (ingredient #1)!
OUTPUT
Encrypted Blob
8f3a9d...
Encrypted Blob
Sent to server


SERVER NEVER SEES THIS
(User's view)
Secret Report.pdf
Password: abc123
8f3a9d2b...#@&%*
8f3a9d2...#@&%*
8f3a9d2...#@&%*
8f3a9d2...#@&%*
8f3a9d2...#@&%*
(Server's view)
token + password shared separately
Secret Report.pdf
Original File
Client Encrypts
Server Stores
Share Link
Recipient Decrypts

Server Database Contents

id	user_id	filename	encrypted_file_path	iv ?	salt ?	password ✗	original_content ✗
1	42	"report.pdf"	"uploads/8f3a9d2b..."	"a3f8c2d1b4e6"	"9b2c4f6e8a1d"	✗	✗
2	43	"photo.jpg"	"uploads/2c6e9f1a..."	"f6b3e8a2c9d4"	"3a7f9c2e6b8d"	✗	✗
✓	✓	✓	✓	✓	✓	✗	✗

Used for decryption on CLIENT side only


TRADITIONAL CLOUD STORAGE
BREAK IN!
✓Your files (readable)
✓Passwords
✓User data
✓Everything
STEALTHSHARE
BLOCKED!
✗Encrypted blobs (useless)
✗Passwords (never stored)
✗File content (can't decrypt)
✓Metadata only (filename, size)